

Module 7

Reinforcement Learning for Nash Differential Games

In this module we present the general solution to the differential non-zero sum Nash game with the conflict of interest game and simultaneous decision making, whose solution was derived in [1]-[2] using dynamic programming of Bellman [3]. The solution to the corresponding linear-quadratic (LQ) Nash differential game problem is presented in detail. It is given in terms of solutions of coupled differential Riccati equations, which in the case of steady state optimization (infinite time optimization period) leads to the coupled algebraic Riccati equations. The algorithm for solving obtained coupled algebraic Riccati equations in terms of solutions of a sequence of decoupled algebraic Lyapunov equations is given according to the result of [4]. The algebraic Lyapunov iterations method of [4] is considered these days as the reinforcement learning (RL) model-based solution to the LQ Nash games using policy iterations as stated in [5].

7.1 Dynamic Programming for Nash Differential Games

Consider a general time-invariant, continuous-time, nonlinear dynamic system controlled by two agents $u_1(t)$ and $u_2(t)$ defined by

$$\frac{dx(t)}{dt} = f(x(t), u_1(t), u_2(t)), \quad x(t_0) = x_0 \quad (7.1)$$

where $f(x(t), u_1(t), u_2(t)) \in \mathbb{R}^n$ is a continuously differentiable vector function of its arguments, $x(t) \in \mathbb{R}^n$ is the system state space vector, $u_1(t) \in \mathbb{R}^{m_1}$, $u_2(t) \in \mathbb{R}^{m_2}$ are the system control inputs (agents), $x(t_0) = x_0$ is the system initial state, and t stands for continuous time with t_0 representing the initial time. The goal of each agent is to minimize its own scalar performance criterion

$$J_i(x(t_0), t_0) = \int_{t_0}^{t_f} g_i(x(\tau), u_1(\tau), u_2(\tau)) d\tau + \gamma_i(x(t_f), t_f), \quad i = 1, 2 \quad (7.2)$$

where $g_i(x(\tau), u_1(\tau), u_2(\tau)), i = 1, 2$ are *nonnegative scalar functions* continuously differentiable with respect to their arguments, representing the value of the performance criterion (cost) at the continuous time instant t (instant cost). t_f is the final time instant assumed to be fixed, and $\gamma_i(x(t_f)), i = 1, 2$ are the final time constraints imposed on the state variables by each agent. Hence, each agent intends to find its minimal performance criterion (minimize its own losses) using its own control action being aware that the other agent is doing the same, that is

$$J_1^{opt}(x(t_0), t_0) = \min_{u_1(t)} \left\{ \int_{t_0}^{t_f} g(x(\tau), u_1(\tau), u_2(\tau)) d\tau + \gamma(x(t_f), t_f) \right\} \quad (7.2a)$$

$$J_2^{opt}(x(t_0), t_0) = \min_{u_2(t)} \left\{ \int_{t_0}^{t_f} g(x(\tau), u_1(\tau), u_2(\tau)) d\tau + \gamma(x(t_f), t_f) \right\} \quad (7.2b)$$

The issue is whether these optimal controls exist, either in the program form (as functions of time, $u_1^{opt}(t), u_2^{opt}(t)$) or the feedback form as functions of state variables that is $u_1(x(t)), u_2(x(t))$, which is particularly important for reinforcement learning. If such a pair of the solutions exists, either in the feedback form or the program form, the solution to the optimization problem (1)-(2) is called the *Nash equilibrium solution*. Mathematically, for two decision makers (agents), the Nash equilibrium is defined by a pair of inequalities

$$\begin{aligned} J_{1Nash}(u_{1Nash}^{opt}, u_{2Nash}^{opt}) &\leq J_{1Nash}(u_1, u_{2Nash}^{opt}) \\ J_{2Nash}(u_{1Nash}^{opt}, u_{2Nash}^{opt}) &\leq J_{2Nash}(u_{1Nash}^{opt}, u_2) \end{aligned} \quad (7.3)$$

The goal is to find the *optimal feedback control* inputs, that minimize the performance criteria defined in (7.2). Assuming that the optimal Nash solution exists, the optimal Nash equilibrium performance criteria are given by

$$J_{iNash}^{opt}(x(t_0), t_0) = \int_{t_0}^{t_f} g_i(x_{Nash}^{opt}(\tau), u_{iNash}^{opt}(\tau), u_{iNash}^{opt}(\tau)) d\tau + \gamma_i(x_{iNash}^{opt}(t_f), t_f), i = 1, 2 \quad (7.4)$$

where the optimal system under the Nash optimal control inputs is defined by

$$\frac{dx_{Nash}^{opt}(t)}{dt} = f(x_{Nash}^{opt}(t), u_{1Nash}^{opt}(t), u_{2Nash}^{opt}(t)), \quad x_{Nash}^{opt}(t_0) = x_0 \quad (7.5)$$

The solution to the dynamic optimization problem defined in (7.1) and (7.2) is based on the *Bellman Optimality Principle* of dynamic programming. Since the optimization tasks are the same for both decision makers (agents, players), we will first apply the dynamic programming procedure to the optimization problem of the first player *assuming that the second player uses the optimal Nash strategy*, and then conclude that the similar result holds for the second player.

Comment 7.1: It is important to emphasize that in theory of Nash games there is a result that shows that if the second player deviates from the optimal Nash strategy (Nash equilibrium) that leaves a room for the first player to deviate from the optimal Nash strategy also and even get a better performance criterion value (smaller loss) than under the optimal Nash strategy. Since the

game is played with simultaneous decision making, it is wise for both players to stick with their optimal Nash strategies. Hence, the Nash strategy is a secure strategy against cheating.

It can be shown by generalizing the result from Module 4 that the optimal performance value for the first player, for the system that in general starts at state $x(t)$ with the initial time t and fixed terminal time t_f , satisfies

$$-\frac{\partial J_{1Nash}^{opt}(x(t),t)}{\partial t} = \min_{u_1(x(t))} \left\{ g_1(x(t), u_1(t), u_{2Nash}^{opt}(t)) + \left(\frac{\partial J_1^{opt}(x(t),t)}{\partial x} \right)^T f(x(t), u_1(t), u_{2Nash}^{opt}(t)) \right\} \quad (7.6a)$$

Equation (7.6) represents the Bellman functional partial differential equation for the first player for the Nash differential game.

Similarly, the second player (decision maker, controller), can assume that the first player uses the optimal control $u_{1Nash}^{opt}(x(t))$ over the entire optimization interval $[t, t_f]$, so that the second player is faced with the classical dynamic optimization problem parametrized by $u_{1Nash}^{opt}(x(t))$. Using the Bellman optimality principle, the Bellman functional equation for the second player for Nash differential games is obtained as follows

$$-\frac{\partial J_{2Nash}^{opt}(x(t),t)}{\partial t} = \min_{u_2(x(t))} \left\{ g_2(x(t), u_{1Nash}^{opt}(t), u_2(t)) + \left(\frac{\partial J_{2Nash}^{opt}(x(t),t)}{\partial x} \right)^T f(x(t), u_{1Nash}^{opt}(t), u_2(t)) \right\} \quad (7.6b)$$

The Hamiltonian of the optimization problem (7.1)-(7.2) are defined by

$$H_1 \left(x(t), u_1(t), u_{2Nash}^{opt}(t), \frac{\partial J_{1Nash}^{opt}(x(t),t)}{\partial x} \right) = g_1(x(t), u_1(t), u_{2Nash}^{opt}(t)) + \left(\frac{\partial J_{1Nash}^{opt}(x(t),t)}{\partial x} \right)^T f(x(t), u_1(t), u_{2Nash}^{opt}(t)) \quad (7.7a)$$

and

$$H_2 \left(x(t), u_{1Nash}^{opt}(t), u_2(t), \frac{\partial J_{2Nash}^{opt}(x(t),t)}{\partial x} \right) = g_2(x(t), u_{1Nash}^{opt}(t), u_2(t)) + \left(\frac{\partial J_{2Nash}^{opt}(x(t),t)}{\partial x} \right)^T f(x(t), u_{1Nash}^{opt}(t), u_2(t)) \quad (7.7b)$$

Using the Hamiltonian notation introduced in (7.7), the *Bellman functional equations for Nash differential games* become

$$-\frac{\partial J_{1Nash}^{opt}(x(t), t)}{\partial t} = \min_{u_1(x(t))} \left\{ H_1 \left(x(t), u_1(t), u_{2Nash}^{opt}(t), \frac{\partial J_{1Nash}^{opt}(x(t), t)}{\partial x} \right) \right\} \quad (7.8a)$$

$$-\frac{\partial J_{2Nash}^{opt}(x(t), t)}{\partial t} = \min_{u_2(x(t))} \left\{ H_2 \left(x(t), u_{1Nash}^{opt}(t), u_2(t), \frac{\partial J_{2Nash}^{opt}(x(t), t)}{\partial x} \right) \right\} \quad (7.8b)$$

Assume that the optimal controls can be uniquely determined (preferably in the feedback form) from (7.8) by performing minimization of the right-hand side of (7.8), using the necessary condition for minimum and solving the corresponding algebraic equations, that is

$$\begin{aligned} \frac{\partial}{\partial u_1} \left\{ H_1 \left(x(t), u_1(t), u_{2Nash}^{opt}(t), \frac{\partial J_{1Nash}^{opt}(x(t), t)}{\partial x} \right) \right\} &= 0 \\ \Rightarrow u_1^{opt} &= u_1^{opt} \left(x_{Nash}^{opt}(t), \frac{\partial J_{1Nash}^{opt}(x(t), t)}{\partial x}, \frac{\partial J_{2Nash}^{opt}(x(t), t)}{\partial x} \right) \\ \frac{\partial}{\partial u_2} \left\{ H_2 \left(x(t), u_{1Nash}^{opt}(t), u_2(t), \frac{\partial J_{2Nash}^{opt}(x(t), t)}{\partial x} \right) \right\} &= 0 \\ \Rightarrow u_2^{opt} &= u_2^{opt} \left(x_{Nash}^{opt}(t), \frac{\partial J_{1Nash}^{opt}(x(t), t)}{\partial x}, \frac{\partial J_{2Nash}^{opt}(x(t), t)}{\partial x} \right) \end{aligned} \quad (7.9)$$

Having obtained the expression for optimal controls in (7.9), the Bellman functional equations of Nash differential games that need to be solved to find the optimal values for the performance criteria (and subsequently the optimal control strategies from (7.9)) become

$$-\frac{\partial J_{1Nash}^{opt}(x(t), t)}{\partial t} = H_1 \left(x_{Nash}^{opt}(t), u_{1Nash}^{opt}(t), u_{2Nash}^{opt}(t), \frac{\partial J_{1Nash}^{opt}(x(t), t)}{\partial x}, \frac{\partial J_{2Nash}^{opt}(x(t), t)}{\partial x} \right) \quad (7.10.a)$$

$$-\frac{\partial J_{2Nash}^{opt}(x(t), t)}{\partial t} = H_2 \left(x_{Nash}^{opt}(t), u_{1Nash}^{opt}(t), u_{2Nash}^{opt}(t), \frac{\partial J_{1Nash}^{opt}(x(t), t)}{\partial x}, \frac{\partial J_{2Nash}^{opt}(x(t), t)}{\partial x} \right) \quad (7.10.b)$$

These are *coupled partial differential equations* that have the form of the *Hamilton-Jacobi-Bellman equation*.

In optimal control theory, the gradient of the optimal performance criterion defines the so-called co-state variables, in this case $p_1(t) = \partial J_1(x(t), t) / \partial x(t)$ and $p_2(t) = \partial J_2(x(t), t) / \partial x(t)$. The differential equations for $p_1(t)$ and $p_2(t)$ can be obtained from (7.10) by taking the partial derivatives with respect to the state variable $x(t)$. By taking the partial derivative for the first player, and using (7.9), we have

$$-\frac{dp_1(t)}{dt} = \frac{\partial H_1}{\partial x} + \left(\frac{\partial u_2}{\partial x} \right)^T \left(\frac{\partial H_1}{\partial u_2} \right), \quad p_1(t_f) = \frac{\partial J_1(x(t_f), t_f)}{\partial x} = \frac{\partial \gamma_1(x(t_f), t_f)}{\partial x} \quad (12a)$$

The co-state equation for the second player can be similarly derived as

$$-\frac{dp_2(t)}{dt} = \frac{\partial H_2}{\partial x} + \left(\frac{\partial u_1}{\partial x} \right)^T \left(\frac{\partial H_2}{\partial u_1} \right), \quad p_2(t_f) = \frac{\partial J_2(x(t_f), t_f)}{\partial x} = \frac{\partial \gamma_2(x(t_f), t_f)}{\partial x} \quad (12b)$$

In summary, for the Nash differential game optimization problem defined by (7.1)-(7.2), the necessary conditions for optimum (Nash equilibrium) represented in terms of Hamiltonians (7.7) are given by

$$\begin{aligned} H_i(x(t), u_1(t), u_2(t), p_i(t)) &= g(x(t), u_1(t), u_2(t)) + p_i^T(t) f(x(t), u_1(t), u_2(t)), \quad i = 1, 2 \\ \frac{dx(t)}{dt} &= \frac{\partial}{\partial p_i} \{ H_i(x(t), u_1(t), u_2(t), p_i(t)) \}, \quad x(t_0) = x_0, \quad i = 1, 2 \\ \frac{dp_1(t)}{dt} &= -\frac{\partial}{\partial x} \{ H_1(x(t), u_1(t), u_2(t), p_1(t)) \} - \left(\frac{\partial u_2}{\partial x} \right)^T \left(\frac{\partial H_1}{\partial u_2} \right), \quad p(t_f) = \frac{\partial J_1(x(t), t)}{\partial x} \Big|_{t=t_f} = \frac{\partial}{\partial x} \{ \gamma_1(x(t_f), t_f) \} \\ \frac{dp_2(t)}{dt} &= -\frac{\partial}{\partial x} \{ H_2(x(t), u_1(t), u_2(t), p_2(t)) \} - \left(\frac{\partial u_1}{\partial x} \right)^T \left(\frac{\partial H_2}{\partial u_1} \right), \quad p(t_f) = \frac{\partial J_2(x(t), t)}{\partial x} \Big|_{t=t_f} = \frac{\partial}{\partial x} \{ \gamma_2(x(t_f), t_f) \} \\ 0 &= \frac{\partial}{\partial u_1} \{ H_1(x(t), u_1(t), u_2(t), p_1(t)) \} \\ 0 &= \frac{\partial}{\partial u_2} \{ H_2(x(t), u_1(t), u_2(t), p_2(t)) \} \end{aligned} \quad (7.13)$$

Comment 7.2: The second term on the right-hand side of the co-state equation disappear in the case the players use the programmed controls, that is $u_1^{opt} = u_1^{opt}(t)$ and $u_2^{opt} = u_2^{opt}(t)$, in which case the corresponding partial derivatives with respect to the state variables are zero. Such a case is not of interest for RL, where controls (actions) are required to be given in terms of state

variables. In the case of feedback controls, the corresponding partial derivatives with respect to the state variables have to be taken. It is also possible that one player uses the feedback control and another one the programmed control, which will appropriately modify the right-hand sides of the co-state differential equations.

Assuming that the unique optimal solutions of (7.9) exist, then using such solutions into the state co-state equations (7.13), a two-point boundary value problem is obtained for finding $x^{opt}(t)$, $p_1^{opt}(t)$, and $p_2^{opt}(t)$

$$\begin{aligned}
 \frac{dx^{opt}(t)}{dt} &= f(x^{opt}(t), p_1^{opt}(t), p_2^{opt}(t)), \quad x^{opt}(t_0) = x_0 \\
 \frac{dp_1^{opt}(t)}{dt} &= -\frac{\partial}{\partial x} \left\{ g_1(x^{opt}(t), p_1^{opt}(t), p_2^{opt}(t)) \right\} - \frac{\partial}{\partial x} \left\{ p_1^{optT}(t) f(x^{opt}(t), p_1^{opt}(t), p_2^{opt}(t)) \right\} \\
 p_1^{opt}(t_f) &= \frac{\partial}{\partial x} \left\{ \gamma_1(x^{opt}(t_f), t_f) \right\} \\
 \frac{dp_2^{opt}(t)}{dt} &= -\frac{\partial}{\partial x} \left\{ g_2(x^{opt}(t), p_1^{opt}(t), p_2^{opt}(t)) \right\} - \frac{\partial}{\partial x} \left\{ p_2^{optT}(t) f(x^{opt}(t), p_1^{opt}(t), p_2^{opt}(t)) \right\} \\
 p_2^{opt}(t_f) &= \frac{\partial}{\partial x} \left\{ \gamma_2(x^{opt}(t_f), t_f) \right\}
 \end{aligned} \tag{9.14}$$

It is not easy to solve this two-point boundary value problem in general. Since we are interested only in the feedback solution $u_1^{opt}(x(t))$ and $u_2^{opt}(x(t))$, the two-point value boundary value problem has to provide analytical solutions for the co-state variable in terms of state variables as $p_1^{opt}(x(t))$ and $p_2^{opt}(x(t))$, which is possible only in some special cases. However, we will see in the follow-up of this section that some approximate iterative techniques of RL, policy iterations or value iterations can produce in the limit the approximate feedback control for some important classes of dynamic systems and performance criteria, especially for the time invariant dynamic systems.

The goal is to obtain the feedback solution (crucial for RL), that is $u_1^{opt}(x(t))$ and $u_2^{opt}(x(t))$, analytical solutions for the co-state variables in terms of state variables are needed as $p_1^{opt}(x(t))$, and $p_2^{opt}(x(t))$. This is possible in the case of a LQ Nash differential game.

7.2 Linear-Quadratic Nash Differential Game

Consider a linear, continuous-time, time-invariant system controlled by two agents (decision makers)

$$\frac{dx(t)}{dt} = Ax(t) + B_1u_1(t) + B_2u_2(t), \quad x(t_0) = x_0 \quad (7.15)$$

where $x(t) \in \mathbb{R}^n$ is the system state space vector, $u_1(t) \in \mathbb{R}^{m_1}$, $u_2(t) \in \mathbb{R}^{m_2}$ are the system control input vectors (control agents), $x(t_0) = x_0$ is the system initial state, and t stands for continuous time with t_0 representing the initial time. Matrices A, B_1, B_2 are constant and of appropriate dimensions.

Scalar quadratic performance criteria, to be minimized, associated with each agent, are defined by

$$J_1(x(t_0)) = \frac{1}{2} \int_{t_0}^{t_f} \left(x^T(t)Q_1x(t) + u_1^T(t)R_{11}u_1(t) + u_2^T(t)R_{12}u_2(t) \right) dt + \frac{1}{2} x^T(t_f)P_{f1}x(t_f) \quad (16.a)$$

$$Q_1 = Q_1^T \geq 0, P_{f1} = P_{f1}^T \geq 0, \quad R_{11} = R_{11}^T > 0, \quad R_{12} = R_{12}^T \geq 0$$

$$J_2(x(t_0)) = \frac{1}{2} \int_{t_0}^{t_f} \left(x^T(t)Q_2x(t) + u_1^T(t)R_{21}u_1(t) + u_2^T(t)R_{22}u_2(t) \right) dt + \frac{1}{2} x^T(t_f)P_{f2}x(t_f) \quad (16.b)$$

$$Q_2 = Q_2^T \geq 0, P_{f2} = P_{f2}^T \geq 0, \quad R_{22} = R_{22}^T > 0, \quad R_{21} = R_{21}^T \geq 0$$

where the scaling factor of 0.5 is introduced for convenience due to the fact that the derivatives of quadratic forms will be taken in the follow-up derivations. Positive semidefinite symmetric matrices Q, P_f, R_{21}, R_{12} and the positive definite symmetric matrices R_{11}, R_{22} are weighted matrices for the state and control variables.

The Hamiltonians defined in the Nash game optimization problem (7.15)-(7.16) are given by

$$H_1(x(t), p_1(t), u_1(t), u_2(t)) = \frac{1}{2} \left(x^T(t)Q_1x(t) + u_1^T(t)R_{11}u_1(t) + u_2^T(t)R_{12}u_2(t) \right) + p_1^T(t) \left(Ax(t) + B_1u_1(t) + B_2u_2(t) \right) \quad (7.17.a)$$

$$H_2(x(t), p_2(t), u_1(t), u_2(t)) = \frac{1}{2} \left(x^T(t)Q_2x(t) + u_1^T(t)R_{21}u_1(t) + u_2^T(t)R_{22}u_2(t) \right) + p_2^T(t) \left(Ax(t) + B_1u_1(t) + B_2u_2(t) \right) \quad (7.17.b)$$

Using the five necessary conditions for optimum defined in (7.13), we have

$$\begin{aligned}
 \frac{dx(t)}{dt} &= Ax(t) + B_1 u_1(t) + B_2 u_2(t), \quad x(t_0) = x_0, \quad i = 1, 2 \\
 \frac{dp_1(t)}{dt} &= -Q_1 x(t) - A^T p_1(t) + \left(\frac{\partial u_2}{\partial x} \right)^T \left(\frac{\partial H_1}{\partial u_2} \right), \quad p_1(t_f) = \frac{\partial J_1(x(t))}{\partial x} \Big|_{t=t_f} = P_{1f} x(t_f) \\
 \frac{dp_2(t)}{dt} &= -Q_2 x(t) - A^T p_2(t) + \left(\frac{\partial u_1}{\partial x} \right)^T \left(\frac{\partial H_2}{\partial u_1} \right), \quad p_2(t_f) = \frac{\partial J_2(x(t))}{\partial x} \Big|_{t=t_f} = P_{2f} x(t_f) \\
 0 &= R_{11} u_1(t) + B_1^T p_1(t) \Rightarrow u_1^{opt}(t) = -R_{11}^{-1} B_1^T p_1(t) \\
 0 &= R_{22} u_2(t) + B_2^T p_2(t) \Rightarrow u_2^{opt}(t) = -R_{22}^{-1} B_2^T p_2(t)
 \end{aligned} \tag{7.18}$$

We have four possibilities for the type of optimal control actions: (a) none of the players uses a feedback strategy in which case both partial derivatives in (2.18) are zero; (b) player one uses feedback so that $\partial u_1(x(t)) / \partial x \neq 0$ and player two uses the open-loop control so that $\partial u_2(t) / \partial x = 0$; (c) player one uses open-loop control so that $\partial u_1(t) / \partial x = 0$ and player two uses feedback control when $\partial u_2(x(t)) / \partial x \neq 0$; and (d) *both players use the feedback strategies, which is particularly important for RL*. In case (d) we will show that there is a linear dependence between the co-state and state variables, that is

$$\begin{aligned}
 p_1 = p_1(x(t)) = K_1(t)x(t) &\Rightarrow u_1 = u_1(x(t)) = -R_{11}^{-1} B_1^T K_1(t)x(t) \Rightarrow \frac{\partial u_1(x(t))}{\partial x} = -R_{11}^{-1} B_1^T K_1(t) \\
 p_2 = p_2(x(t)) = K_2(t)x(t) &\Rightarrow u_2 = u_2(x(t)) = -R_{22}^{-1} B_2^T K_2(t)x(t) \Rightarrow \frac{\partial u_2(x(t))}{\partial x} = -R_{22}^{-1} B_2^T K_2(t)
 \end{aligned} \tag{7.19}$$

Since in addition the partial derivatives of the Hamiltonians are given by

$$\begin{aligned}
 \frac{\partial H_1}{\partial u_2} &= R_{12} u_2 + B_2^T p_1 = -R_{12} R_{22}^{-1} B_2^T K_2(t)x(t) + B_2^T p_1 \\
 \frac{\partial H_2}{\partial u_1} &= R_{21} u_1 + B_1^T p_2 = -R_{21} R_{11}^{-1} B_1^T K_1(t)x(t) + B_1^T p_2
 \end{aligned} \tag{7.20}$$

the state co-state equations in the case when both players use feedback controls become

$$\begin{aligned}
 \frac{dx(t)}{dt} &= (A - S_1 K_1(t) - S_2 K_2(t))x(t), & x(t_0) &= x_0 \\
 \frac{dp_1(t)}{dt} &= -(Q_1 + K_2(t)Z_2K_2(t))x(t) - (A - S_2K_2(t))^T p_1(t), & p_1(t_f) &= P_{1f}x(t_f) \\
 \frac{dp_2(t)}{dt} &= -(Q_2 + K_1(t)Z_1K_1(t))x(t) - (A - S_1K_1(t))^T p_2(t), & p_2(t_f) &= P_{2f}x(t_f)
 \end{aligned} \tag{7.21}$$

where

$$S_1 = B_1 R_{11}^{-1} B_1^T, \quad S_2 = B_2 R_{22}^{-1} B_2^T, \quad Z_1 = B_1 R_{11}^{-1} R_{21} R_{11}^{-1} B_1^T, \quad Z_2 = B_2 R_{22}^{-1} R_{12} R_{22}^{-1} B_2^T \tag{7.22}$$

Using the relationships between the state and co-state variables stated in (7.19) and the state co-state equations (7.21), after some algebra it can be shown that matrices $K_1(t)$ and $K_2(t)$ satisfy the following coupled differential Riccati equations

$$\begin{aligned}
 -\frac{dK_1(t)}{dt} &= K_1(t)A + A^T K_1(t) + Q_1 - K_1(t)S_1 K_1(t) \\
 &\quad - K_1(t)S_2 K_2(t) - K_2(t)S_2 K_1(t) + K_2(t)Z_2 K_2(t), \quad K_1(t_f) = P_{1f} \\
 -\frac{dK_2(t)}{dt} &= K_2(t)A + A^T K_2(t) + Q_2 - K_2(t)S_2 K_2(t) \\
 &\quad - K_2(t)S_1 K_1(t) - K_1(t)S_1 K_2(t) + K_1(t)Z_1 K_1(t), \quad K_2(t_f) = P_{2f}
 \end{aligned} \tag{7.23}$$

These coupled differential equations can be solved numerically integrating backwards in time. Since $K_1(t)$ and $K_2(t)$ vary in time the closed-loop optimal system will be time varying. If we extend the optimization period to infinity, $t_f \rightarrow \infty$, the coupled differential Riccati equations (7.23), under certain linear system controllability and observability conditions, become the coupled algebraic Riccati equations of Nash differential games

$$\begin{aligned}
 0 &= K_1 A + A^T K_1 + Q_1 - K_1 S_1 K_1 - K_1 S_2 K_2 - K_2 S_2 K_1 + K_2 Z_2 K_2 \\
 0 &= K_2 A + A^T K_2 + Q_2 - K_2 S_2 K_2 - K_2 S_1 K_1 - K_1 S_1 K_2 + K_1 Z_1 K_1
 \end{aligned} \tag{7.24}$$

These coupled algebraic Riccati equations are important for RL. They have many solutions, but under controllability and observability conditions, the unique positive definite stabilizing solutions exist. In the next section, we will see how they can be solved using the policy iterations in terms of decoupled algebraic Lyapunov equations.

Having obtained the solutions for K_1, K_2 , the optimal feedback controls can be now expressed in the feedback form

$$\begin{aligned} u_1^{opt}(x(t)) &= -R_1^{-1} B_1^T K_1 x^{opt}(t) = -F_1^{opt} x^{opt}(t) \\ u_2^{opt}(x(t)) &= -R_2^{-1} B_2^T K_2 x^{opt}(t) = -F_2^{opt} x^{opt}(t) \end{aligned} \quad (7.25)$$

where the optimal system dynamic equation is given by

$$\begin{aligned} \frac{dx^{opt}(t)}{dt} &= (A - B_1 F_1^{opt}(t) + B_2 F_2^{opt}(t)) x^{opt}(t) = (A - S_1 K_1 - S_2 K_2) x^{opt}(t), \quad x^{opt}(t_0) = x_0 \\ \Rightarrow x^{opt}(t) &= e^{(A - S_1 K_1 - S_2 K_2)(t - t_0)} x_0 \end{aligned} \quad (7.26)$$

In order to find the optimal performance for each player we substitute their optimal controls into (7.16) with $t_f \rightarrow \infty$, which leads to

$$J_1^{opt}(x(t_0)) = \frac{1}{2} \int_{t_0}^{\infty} \left(x^{optT}(t) (Q_1 + K_1 S_1 K_1 + K_2 Z_2 K_2) x(t) \right) dt \quad (7.27.a)$$

$$J_2^{opt}(x(t_0)) = \frac{1}{2} \int_{t_0}^{\infty} \left(x^{optT}(t) (Q_2 + K_2 S_2 K_2 + K_1 Z_1 K_1) x(t) \right) dt \quad (7.27.b)$$

Eliminating $x^{opt}(t)$ from (7.27) using (7.26), we obtain

$$J_1^{opt}(x(t_0)) = \frac{1}{2} x_0^T \int_{t_0}^{\infty} \left(e^{(A - S_1 K_1 - S_2 K_2)^T (t - t_0)} (Q_1 + K_1 S_1 K_1 + K_2 Z_2 K_2) e^{(A - S_1 K_1 - S_2 K_2)(t - t_0)} \right) dt x_0 \quad (7.28.a)$$

$$J_2^{opt}(x(t_0)) = \frac{1}{2} x_0^T \int_{t_0}^{\infty} \left(e^{(A - S_1 K_1 - S_2 K_2)^T (t - t_0)} (Q_2 + K_2 S_2 K_2 + K_1 Z_1 K_1) e^{(A - S_1 K_1 - S_2 K_2)(t - t_0)} \right) dt x_0 \quad (7.28.b)$$

The above integrals can be evaluated using the solution of the corresponding Lyapunov equations

$$(A - S_1 K_1 - S_2 K_2)^T V_1 + V_1 (A - S_1 K_1 - S_2 K_2) + Q_1 + K_1 S_1 K_1 + K_2 Z_2 K_2 = 0$$

$$(A - S_1 K_1 - S_2 K_2)^T V_2 + V_2 (A - S_1 K_1 - S_2 K_2) + Q_2 + K_2 S_2 K_2 + K_1 Z_1 K_1 = 0$$

Moreover, by setting $V_1 = K_1$ and $V_2 = K_2$, we see that V_1 and V_2 satisfy the coupled algebraic Riccati equations (7.24), so that the optimal performance criteria are given by

$$J_1^{opt}(x(t_0)) = \frac{1}{2} x_0^T V_1 x_0 = \frac{1}{2} x_0^T K_1 x_0, \quad J_2^{opt}(x(t_0)) = \frac{1}{2} x_0^T V_2 x_0 = \frac{1}{2} x_0^T K_2 x_0 \quad (7.29)$$

These kinds of optimization problems with conflict of interest appear in power systems, machine learning, economics, aerospace, and other engineering and scientific areas. In the past ten fifteen year, quite a lot of papers on Nash differential games and their applications to reinforcement learning were published, see for example references [6]-[19].

7.3 Lyapunov Iterations for the Coupled Nash Algebraic Riccati Equations

It is known in RL that the coupled algebraic Riccati equations can be solved via the algebraic Lyapunov iterations using approximate dynamic programming policy iterations [4]

$$\left(A - S_1 K_1^{(i)} - S_2 K_2^{(i)} \right)^T K_1^{(i+1)} + K_1^{(i+1)} \left(A - S_1 K_1^{(i)} - S_2 K_2^{(i)} \right) + Q_1 + K_1^{(i)} S_1 K_1^{(i)} + K_2^{(i)} Z_2 K_2^{(i)} = 0 \quad (7.30a)$$

$$\left(A - S_1 K_1^{(i)} - S_2 K_2^{(i)} \right)^T K_2^{(i+1)} + K_2^{(i+1)} \left(A - S_1 K_1^{(i)} - S_2 K_2^{(i)} \right) + Q_2 + K_2^{(i)} S_2 K_2^{(i)} + K_1^{(i)} Z_1 K_1^{(i)} = 0 \quad (7.30b)$$

where the convergence to the positive semi-stabilizing solutions of these equations was established under assumptions of system controllability (stabilizability) and observability (detectability).

Assumption 7.1: Either the triple $(A, B_1, \text{Chol}(Q_1))$ or $(A, B_2, \text{Chol}(Q_2))$ is controllable (stabilizable) and observable (detectable).

Comment 7.3: It looks like this is a generalized Kleinman algorithm [20], which is fact the Newton method for solving the algebraic Riccati equation. However, it can be shown that this is not a Newton algorithm. The Newton algorithms are known for their quadratic convergence rate in the neighborhood of the sought solution. The presented algorithm (7.30) has the convergence rate that is in between of linear and quadratic.

The algorithm requires that the initial feedback gains are stabilizing, that is $u_1^{(0)}(x(t)) = -F_1^{(0)}x(t)$ and $u_2^{(0)}(x(t)) = -F_2^{(0)}x(t)$, make the system feedback matrix $A - B_1 F_1^{(0)} - B_2 F_2^{(0)} = A - S_1 K_1^{(0)} - S_2 K_2^{(0)}$ asymptotically stable. Such an asymptotically stable initialization procedure was also proposed in [4]. Assume that the triple $(A, B_1, \text{Chol}(Q_1))$ is stabilizable detectable, then the solutions of these two standard algebraic Riccati equations (ARE)

$$\begin{aligned} K_1^{(0)} A + A^T K_1^{(0)} + Q_1 - K_1^{(0)} S_1 K_1^{(0)} &= 0 \\ K_2^{(0)} (A - S_1 K_1^{(0)}) + (A - S_1 K_1^{(0)})^T K_2^{(0)} + Q_2 - K_2^{(0)} S_2 K_2^{(0)} &= 0 \end{aligned} \quad (7.31)$$

make the system feedback matrix $A - S_1 K_1^{(0)} - S_2 K_2^{(0)}$ asymptotically stable. Namely, under the given assumption, the solution of the first standard ARE $K_1^{(0)}$ makes the system feedback matrix

$A - S_1 K_1^{(0)}$ asymptotically stable. Since $A - S_1 K_1^{(0)}$ plays the role of the system matrix in the second standard ARE. Due to its asymptotic stability, the assumption of stabilizability-detectability is satisfied, so that the second standard ARE produces stabilizing feedback solution $K_2^{(0)}$ for $A - S_1 K_1^{(0)}$ so that $(A - S_1 K_1^{(0)}) - S_2 K_2^{(0)}$ is asymptotically stable.

The policy iterations algorithm (7.30) can be also presented in terms of the feedback gains

$$F_1^{(i)} = R_{11}^{-1} B_1^T K_1^{(i)}, \quad F_2^{(i)} = R_{22}^{-1} B_2^T K_2^{(i)} \quad (7.32)$$

by replacing $S_1 K_1^{(i)}$ by $B_1 F_1^{(i)}$, $S_2 K_2^{(i)}$ by $B_2 F_2^{(i)}$, $K_1^{(i)} Z_1 K_1^{(i)}$ by $F_1^{(i)T} R_{11} F_1^{(i)}$, and $K_2^{(i)} Z_2 K_2^{(i)}$ by $F_2^{(i)T} R_{22} F_2^{(i)}$, note the definitions of $S_{ij}, i, j = 1, 2$ and $Z_i, i = 1, 2$ matrices from (7.22). This leads to

$$(A - B_1 F_1^{(i)} - B_2 F_2^{(i)})^T K_1^{(i+1)} + K_1^{(i+1)} (A - B_1 F_1^{(i)} - B_2 F_2^{(i)}) + Q_1 + F_1^{(i)T} R_{11} F_1^{(i)} + F_2^{(i)T} R_{21} F_2^{(i)} = 0 \quad (7.33a)$$

$$(A - B_1 F_1^{(i)} - B_2 F_2^{(i)})^T K_2^{(i+1)} + K_2^{(i+1)} (A - B_1 F_1^{(i)} - B_2 F_2^{(i)}) + Q_2 + F_2^{(i)T} R_{22} F_2^{(i)} + F_1^{(i)T} R_{12} F_1^{(i)} = 0 \quad (7.33b)$$

Algorithm (7.33) appears to be more in the spirit of RL than algorithm (7.30), since the feedback gains are required for interactions with the system.

MATLAB Example 7.1:

```
% Example 2, Li et al, Science China: Information Science, % 62:Art.
no. 52204, 2019
A = [-0.0366    0.0271    0.0188   -0.4555;
      0.0482   -1.01      0.0024   -4.0208;
      0.1002    0.2855   -0.707     1.3229;
      0         0         1         0]
B1 = [0.4422; 3.0447; -5.52; 0]
B2 = [0.1761; -7.5922; 4.99; 0]
x0 = [10; 20; 15; 5]
R11 = 1; R12 = 0.25; R21 = 0.6; R22 = 2;
Q1 = diag([3.5, 2, 4, 5]); Q2 = diag([1.5, 6, 3, 1]);
S1=B1*inv(R11)*B1'; S2=B2*inv(R22)*B2';
Z1=B1*inv(R11)*R21*inv(R11)*B1'; Z2=B2*inv(R22)*R12*inv(R22)*B2';
% Li-Gajic algorithm
% Initialization
P10=are(A,S1,Q1); P20=are(A-S1*P10,S2,Q2+P10*Z1*P10);
% Lyapunov Policy Iterations
for i=1:10
    P1i=lyap((A-S1*P10-S2*P20)',Q1+P10*S1*P10+P20*Z2*P20);
```

REINFORCEMENT LEARNING FOR ENGINEERING MATLAB DESIGN PROJECTS

```

P2i=lyap((A-S1*P10-S2*P20)',Q2+P20*S2*P20+P10*Z1*P10);
P10=P1i;
P20=P2i;
J1(i)=trace(P1i);
J2(i)=trace(P2i);
end
error1=norm(P1i*A+A'*P1i+Q1-P1i*S1*P1i-P2i*S2*P1i-...
P1i*S2*P2i+P2i*Z2*P2i)
error2=norm(P2i*A+A'*P2i+Q2-P2i*S2*P2i-P2i*S1*P1i-...
P1i*S1*P2i+P1i*Z1*P1i)
figure (1)
plot(J1); grid; xlabel('Iterations');
ylabel('Approximate Performance Criterion J1')
figure (2)
plot(J2); grid; xlabel('Iterations');
ylabel('Approximate Performance Criterion J2')
% It is nicer if the results for approximate criteria are      % presented
in a table.

```

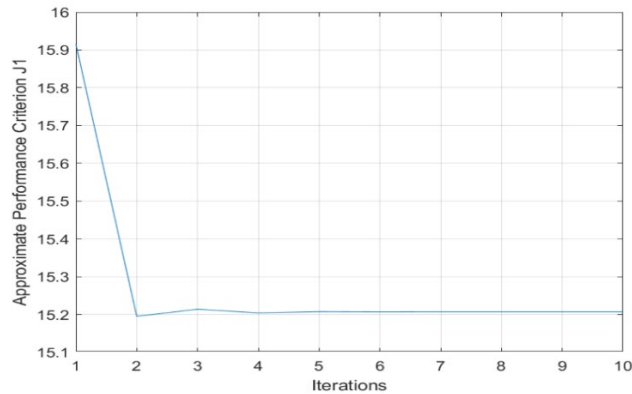
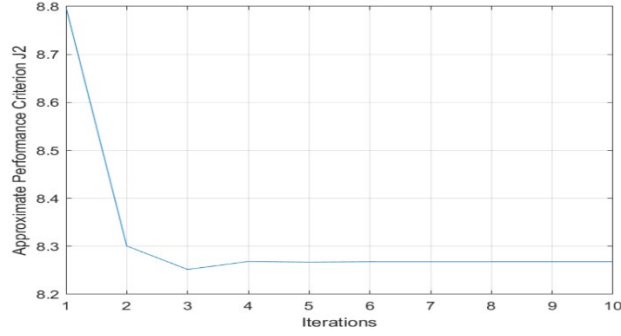


Figure 7.1: Approximate performance criterion $J_1^{(i)}$ per iteration


 Figure 7.2: Approximate performance criterion $J_2^{(i)}$ per iteration

The approximate performance criteria are also presented in Table 7.1, which provides more precise reading of the decimal digits.

i	$J_1^{(i)}$	$J_2^{(i)}$
1	15.9145	8.7983
2	15.1950	8.3006
3	15.2139	8.2517
4	15.2041	8.2683
5	15.2074	8.2669
6	15.2068	8.2678
7	15.2071	8.2677
8	15.2070	$8.2678 = J_2^{opt}$
9	$15.2071 = J_1^{opt}$	8.2678
10	15.2071	8.2678

Table 7.1: Approximate performance criteria per iteration

It can be seen from this table and Figures 7.1 and 7.2 that the convergence is “slightly” non monotonic due to the impact of the opposite player, which is in spirit of Nash differential games, which are based on basic assumption that the game is with conflict of interest and simultaneous decision making.

7.4 RL Policy Iterations for LQ Nash Differential Games Project Demo

In this section we consider the leader-follower vehicle tracking system from [20], which has been also considered in a very recent paper [22]. The system matrices and the performance criterion weighted matrices are presented in the follow-up MATLAB script. The approximate and optimal state trajectories were generated by using Simulink whose block diagram is presented in Figure 7.3. It can be observed from presented Figures 7.4-7.6, pretty fast convergence of the approximate state trajectories $x_3(t), x_5(t), x_6(t)$ to their optimal ones. Approximate and optimal control trajectories are presented in Figures 7.7-7.8 showing pretty rapid convergence to the optimal control inputs (agent actions). Approximate and optimal performance criteria are presented in Figures 7.9-7.10, showing rapid convergence to the optimal values, and indicating that the convergence is not monotonic. Here, we present the solution via the MATLAB m-file. The MATLAB live script, mlx-file solution is presented afterwords.

MATLAB RL-Design-Project Program 7.1: (m-file using Simulink)

```
clear all;
close all; clc
% Leader-follower vehicle tracking system, L. Xu (2024)
A=[0 1 0 0 0 0;
   -0.1326 -0.3931 0.0983 -0.2806 -1.2335 0.1064;
   0 0 0 1 0 0;
   -8.1668 -1.7806 0.3826 -32.667 -0.0266 2.534;
   0 0 0 0 0 1;
   0.0667 0.0085 -0.0021 0.2667 0 -0.0272];
B1=[0 -0.992 0 0.0316 0 -0.0794]';
B2=[0 -0.992 0 0.0316 0 0.0794]';
x0 = [1 0 0 0 0 0]';
R11=5; R22=5; R12=2; R21=2;
R1I=inv(R11); R2I=inv(R22);
Q1=eye(6); Q2=eye(6);
S1=B1*R1I*B1'; S2=B2*R2I*B2';
Z1=B1*R1I*R21*R1I*B1'; Z2=B2*R2I*R12*R2I*B2';
% Li-Gajic Algorithm Initialization
P10=are(A,S1,Q1); P20=are(A-S1*P10,S2,Q2+P10*Z1*P10);
% Lyapunov Policy Iterations
iter = 13
for i=1:1:iter
P1i=lyap((A-S1*P10-S2*P20)',Q1+P10*S1*P10+P20*Z2*P20);
P2i=lyap((A-S1*P10-S2*P20)',Q2+P20*S2*P20+P10*Z1*P10);
```

REINFORCEMENT LEARNING FOR NASH DIFFERENTIAL GAMES

```

F1i=R1I*B1'*P1i;F2i=R2I*B2'*P2i;
P10=P1i; P20=P2i;
J1(i)=0.5*x0'*P1i*x0;
% Calling SIMULINK (interacting with environment)
model=sim(vehicle_tracking_system_nash_game)
% Collecting SIMULINK results
    time_itr(1:length(model.t),i) = model.t;
    X_1(1:length(model.t),i) = model.x(:,1);
    X_2(1:length(model.t),i) = model.x(:,2);
    X_3(1:length(model.t),i) = model.x(:,3);
    X_4(1:length(model.t),i) = model.x(:,4);
    X_5(1:length(model.t),i) = model.x(:,5);
    X_6(1:length(model.t),i) = model.x(:,6);
U_1(1:length(model.t),i,1:size(model.u1(:,:),2))=model.u1(:,:);
U_2(1:length(model.t),i,1:size(model.u2(:,:),2))=model.u2(:,:);
end
% Plotting approximate and optimal states
markers = {'o','+','*','.', 'x','s','d','^','v','>','<','p','h'};
lineStyles = {'-','--',':', '-.'};
    figure (3)
    for i=1:iter
hold on
Iname_3 = sprintf('Iteration %d', i);
plot(time_itr(:,i),X_3(:,i),'Marker',markers{mod(i,iter+1)},...
      'linestyle',lineStyles{3},'DisplayName',Iname_3)
    end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_3')
    figure (5)
    for i=1:iter
hold on
Iname_5 = sprintf('Iteration %d', i); plot(time_itr(:,i),X_5(:,i),...
'Marker',markers{mod(i,iter+1)},'linestyle',lineStyles{2},...
'DisplayName',Iname_5)
    end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_5')
    figure (6)
    for i=1:iter

```

REINFORCEMENT LEARNING FOR ENGINEERING MATLAB DESIGN PROJECTS

```

hold on
Iname_6 = sprintf('Iteration %d', i); plot(time_itr(:,i),X_6(:,i),...
'Marker',markers{mod(i,iter+1)},'linestyle',lineStyles{2},...
'DisplayName',Iname_6)
    end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_6')
% Plotting approximate optimal controls (actions)
figure (7)
for i=1:iter
Iname_u1 = sprintf('Iteration %d', i);
plot(time_itr(:,i),U_1(:,i),'Marker',markers{mod(i,iter+1)},...
'linestyle',lineStyles{max(mod(i,2),1)},'DisplayName',Iname_u1)
hold on
    end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('control u_1(t)')
figure (8)
for i=1:iter
Iname_u2 = sprintf('Iteration %d', i);
plot(time_itr(:,i),U_2(:,i),'Marker',markers{mod(i,iter+1)},...
'linestyle',lineStyles{max(mod(i,2),1)},'DisplayName',Iname_u2)
hold on
    end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('control u_2(t)')
% Solutions accuracy
i
error1=norm(P1i*A+A'*P1i+Q1-P1i*S1*P1i-P2i*S2*P1i-...
P1i*S2*P2i+P2i*Z2*P2i)
error2=norm(P2i*A+A'*P2i+Q2-P2i*S2*P2i-P2i*S1*P1i-...
P1i*S1*P2i+P1i*Z1*P1i)
% Plots of approximate performance criteria
figure (9)
plot(J1); grid; xlabel('Iterations');
ylabel('Approximate Performance Criterion J1')
figure (10)

```

REINFORCEMENT LEARNING FOR NASH DIFFERENTIAL GAMES

```
plot(J2); grid; xlabel('Iterations');  
ylabel('Approximate Performance Criterion J2')
```

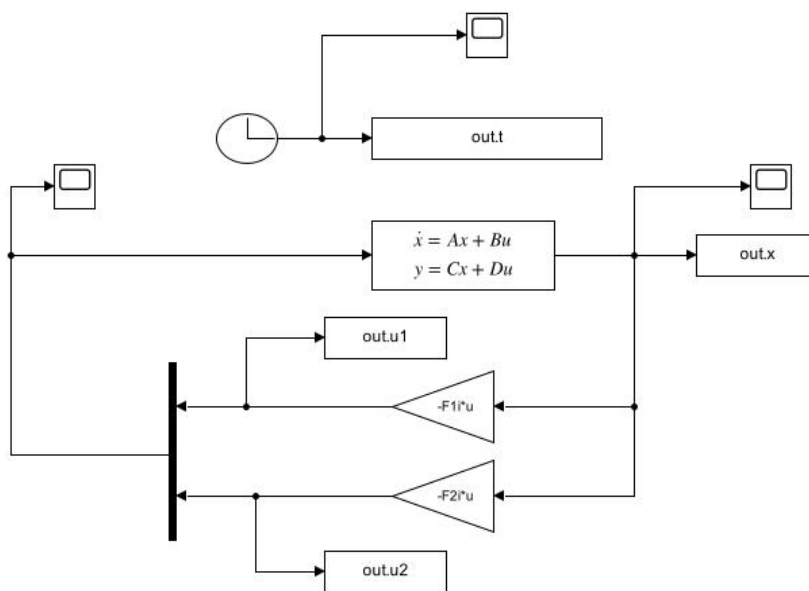


Figure 7.3: Simulink block diagram used in RL for Nash game

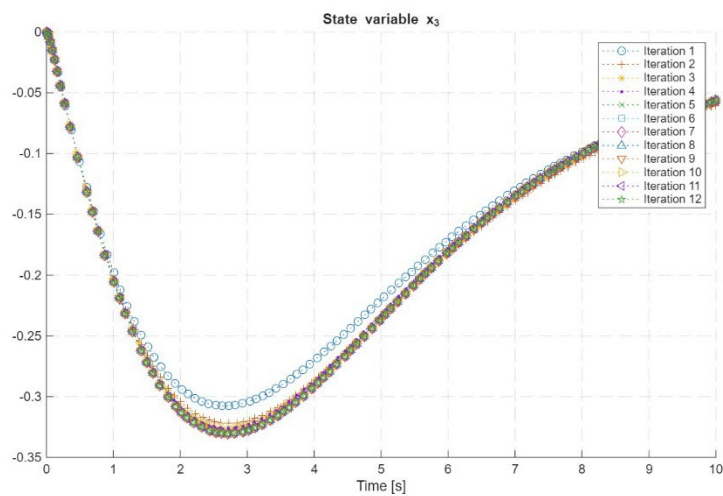


Figure 7.4: Approximate and optimal trajectories for $x_3(t)$

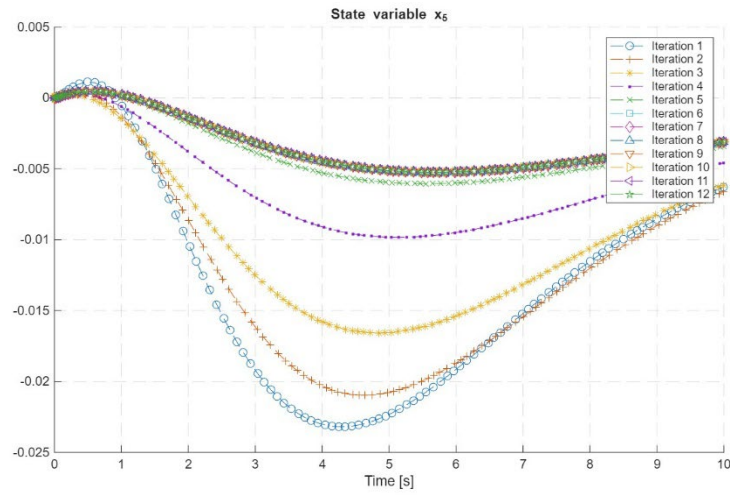


Figure 7.5: Approximate and optimal trajectories for $x_5(t)$

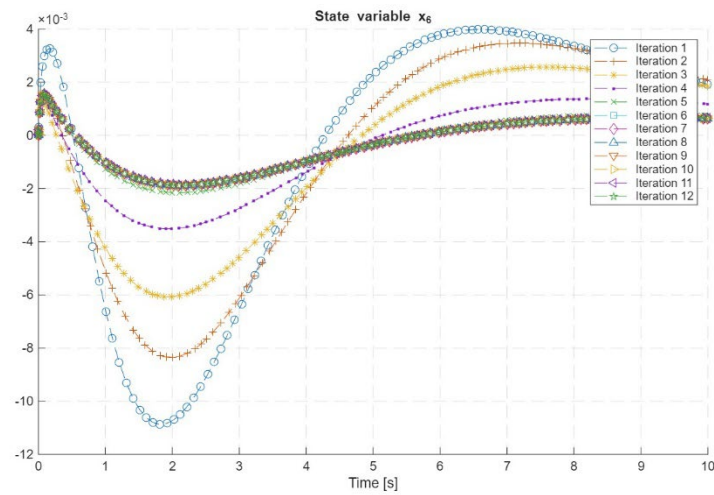


Figure 7.6: Approximate and optimal trajectories for $x_6(t)$

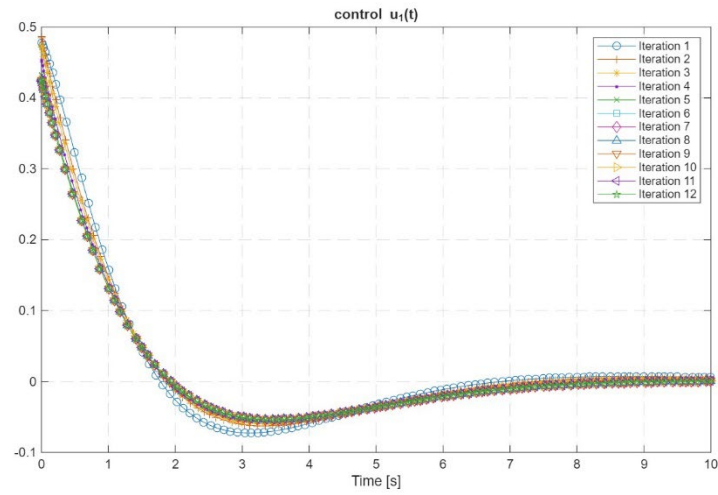


Figure 7.7: Approximate and optimal control $u_1(t)$

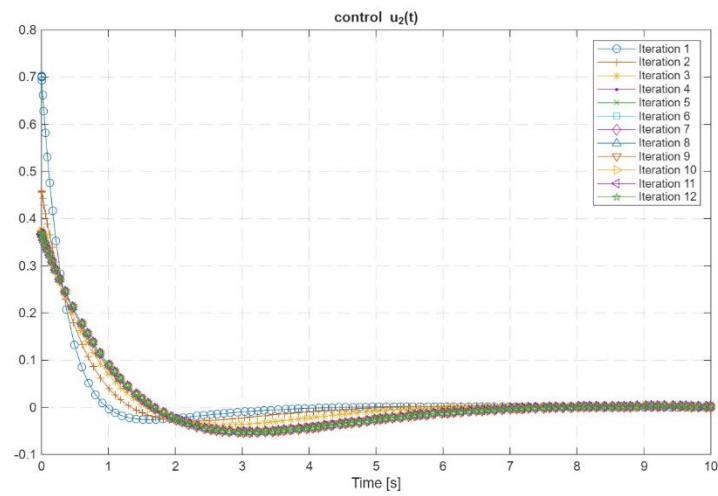


Figure 7.8: Approximate and optimal control $u_2(t)$

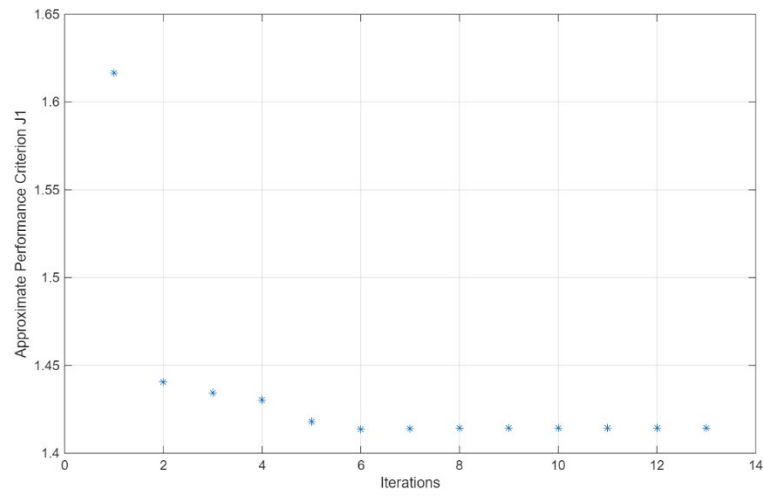


Figure 7.9: Approximate and optimal performance criterion J_1 values

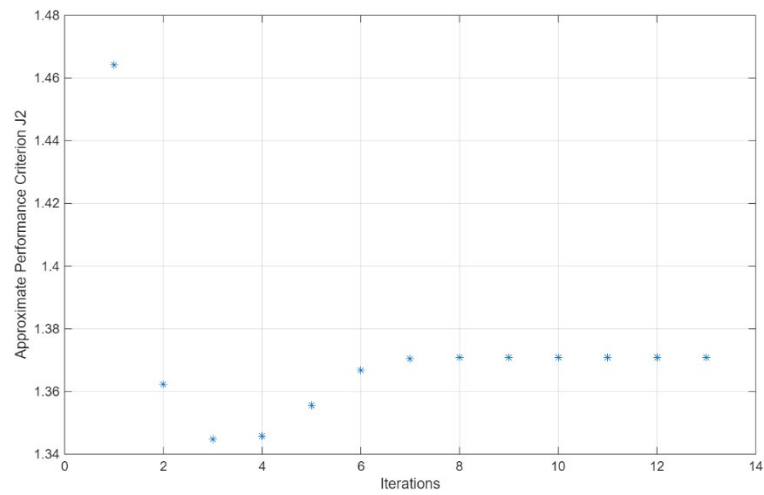


Figure 7.10: Approximate and optimal performance criterion J_2 values

MATLAB RL-Design-Project Program 7.1: (mlx-file using Simulink)

The interplay of the MATLAB live script and Simulink, for this particular example, by running the program: MATLAB RL-Design-Project Program 7.1.mlx in the live editor mode, using the MATLAB Live Editor “export” function is presented below. Export of data was done via the Microsoft word file, and the corresponding file is lightly edited. Since the MATLAB code is presented on the previous pages, the code was hidden.

Linear-Quadratic Nash Differential Games

Consider a linear, continuous-time, time-invariant system controlled by two agents (decision makers)

$$\frac{dx(t)}{dt} = Ax(t) + B_1u_1(t) + B_2u_2(t), \quad x(t_0) = x_0$$

where $x(t) \in \mathbb{R}^n$ the system state space vector, $u_1(t) \in \mathbb{R}^{m_1}$, $u_2(t) \in \mathbb{R}^{m_2}$ system control input vectors (control agents), $x(t_0) = x_0$ is the system initial state, and t stands for continuous time with t_0 representing the initial time. Matrices A, B_1, B_2 are constant and of appropriate dimensions. Scalar quadratic performance criteria, to be minimized, associated with each agent, are defined by

$$\begin{aligned} J_1(x(t_0)) &= \frac{1}{2} \int_{t_0}^{\infty} \left(x^T(t)Q_1x(t) + u_1^T(t)R_{11}u_1(t) + u_2^T(t)R_{12}u_2(t) \right) dt \\ J_2(x(t_0)) &= \frac{1}{2} \int_{t_0}^{\infty} \left(x^T(t)Q_2x(t) + u_1^T(t)R_{21}u_1(t) + u_2^T(t)R_{22}u_2(t) \right) dt \\ Q_1 = Q_1^T &\geq 0, \quad R_{11} = R_{11}^T > 0, \quad R_{12} = R_{12}^T \geq 0 \\ Q_2 = Q_2^T &\geq 0, \quad R_{22} = R_{22}^T > 0, \quad R_{21} = R_{21}^T \geq 0 \end{aligned}$$

where the scaling factor of 0.5 is introduced for convenience due to the fact that the derivatives of quadratic forms will be taken in the follow-up derivations. Positive semidefinite symmetric matrices Q, R_{21}, R_{12} and the positive definite symmetric matrices R_{11}, R_{22} are weighted matrices for the state and control variables.

Coupled algebraic Riccati equations of Nash differential games:

$$\begin{aligned} 0 &= K_1A + A^TK_1 + Q_1 - K_1S_1K_1 - K_1S_2K_2 - K_2S_2K_1 + K_2Z_2K_2 \\ 0 &= K_2A + A^TK_2 + Q_2 - K_2S_2K_2 - K_2S_1K_1 - K_1S_1K_2 + K_1Z_1K_1 \end{aligned}$$

Lyapunov Iterations for the Coupled Nash Algebraic Riccati Equations

It is known in the reinforcement learning that the coupled algebraic Riccati equations can be solved via the algebraic Lyapunov iterations using approximate dynamic programming policy iterations

$$(A - S_1 K_1^{(i)} - S_2 K_2^{(i)})^T K_1^{(i+1)} + K_1^{(i+1)} (A - S_1 K_1^{(i)} - S_2 K_2^{(i)}) + Q_1 + K_1^{(i)} S_1 K_1^{(i)} + K_2^{(i)} Z_2 K_2^{(i)} = 0$$

$$(A - S_1 K_1^{(i)} - S_2 K_2^{(i)})^T K_2^{(i+1)} + K_2^{(i+1)} (A - S_1 K_1^{(i)} - S_2 K_2^{(i)}) + Q_2 + K_2^{(i)} S_2 K_2^{(i)} + K_1^{(i)} Z_1 K_1^{(i)} = 0$$

where the convergence to the positive semi-stabilizing solutions of these equations was established under assumptions of system controllability (stabilizability) and observability (detectability).

Assumption: Either the triple $(A, B_1, \text{Chol}(Q_1))$ or $(A, B_2, \text{Chol}(Q_2))$ is controllable (stabilizable) and observable (detectable).

The algorithm requires that the initial feedback gains are stabilizing, that is $u_1^{(0)}(x(t)) = -F_1^{(0)}x(t)$ and $u_2^{(0)}(x(t)) = -F_2^{(0)}x(t)$, make the system feedback matrix $A - B_1 F_1^{(0)} - B_2 F_2^{(0)} = A - S_1 K_1^{(0)} - S_2 K_2^{(0)}$ asymptotically stable. Such an asymptotically stable initialization procedure is proposed in [1]. Assume that the triple $(A, B_1, \text{Chol}(Q_1))$ is stabilizable detectable, then the solutions of these two standard algebraic Riccati equations (ARE)

$$K_1^{(0)} A + A^T K_1^{(0)} + Q_1 - K_1^{(0)} S_1 K_1^{(0)} = 0$$

$$K_2^{(0)} (A - S_1 K_1^{(0)}) + (A - S_1 K_1^{(0)})^T K_2^{(0)} + Q_2 - K_2^{(0)} S_2 K_2^{(0)} = 0$$

Reinforcement Learning Policy Iterations for Nash Differential Games

The topic is the reinforcement learning policy iterations for the linear-quadratic non-zero-sum (conflict of interest) Nash differential games. Students should master Chapter (Module) 7 of *Reinforcement Learning for Engineering MATLAB Design Projects*, by L. Xu and Z. Gajic, where the corresponding MATLAB Design Project Demo is presented in detail.

Project Report Format: The project reports are supposed to be typed and contain standard technical paper parts: introduction, problem formulation, main results, conclusions, references. The project report should include obtained simulation results, tables, and figures. The program code should be placed at the end in the report as an appendix. The Simulink block diagram should be included in the main body of the project report.

Project Requirements: Implement the policy iteration algorithm for non-zero sum Nash differential games from reference [1]. Good system model to be simulated is given on page 325 of the book chapter by Vrabie and Lewis 2012, reference [2]. An interesting example was

considered in [3] "using microsatellites to implement attitude takeover control for failed spacecraft" with three control agents.

Present two figures and one table. Plot on one figure selected approximates feedback controls obtained for $k = 1, 2, 3, 4, 5$, (or other equally spaced iterations in the case of slow convergence) and on another figure plot the approximate trajectories for a selected (the one that performs the worst in your opinion) state variable for $k = 1, 2, 3, 4, 5$ (or other equally spaced iterations in the case of slow convergence). In the tables provide the values of the performance criteria per iteration, showing the convergence to the optimal values.

To complete the project task students must use Simulink to simulate system (environment) dynamics and interactions of the agents with the system. Upon successful completion with the MATLAB script (m-file) convert the MATLAB script (m-file) into the MATLAB live script (mlx-file). See instructions how to convert an m-file into an lxm-file from Chapter 2, pages 33-24, of Chapter 7, of *Reinforcement Learning for Engineering MATLAB Design Projects*, by L. Xu and Z. Gajic. Run the obtained mlx-file and present the corresponding automatically generated results.

Reference:

1. T. Y. Li and Z. Gajic, "Lyapunov iterations for solving coupled algebraic Riccati equations of Nash differential games and the algebraic Riccati equation of zero-sum games," p. 333-351, in *New Trends in Dynamic Games and Applications*, G. J. Olsder (ed.), Birkhauser, 1995
2. D. Vrabie and F. Lewis, "Integral Reinforcement Learning for Finding Online the Feedback Nash Equilibrium of Nonzero-Sum Differential Games," 313-330, Chapter 17 in *Advances in Reinforcement Learning*, A. Mellouk (ed.), IntechChina, 2012.
3. Y. Chai, J. Luo, N. Han, and J. Xie, "Linear quadratic differential game approach for attitude takeover control of failed spacecraft," *Acta Astronautica*, Vol. 175, 142-154, 2020.

Model

A = 6×6

0	1.0000	0	0	0	0
-0.1326	-0.3931	0.0983	-0.2806	-1.2335	0.1064
0	0	0	1.0000	0	0
-8.1668	-1.7806	0.3826	-32.6670	-0.0266	2.5340
0	0	0	0	0	1.0000
0.0667	0.0085	-0.0021	0.2667	0	-0.0272

B1 = 6×1

0
-0.9920
0
0.0316

REINFORCEMENT LEARNING FOR ENGINEERING MATLAB DESIGN PROJECTS

```
0
-0.0794
B2 = 6×1
0
-0.9920
0
0.0316
0
0.0794
```

```
x0 = 6×1
1
0
0
0
0
0
0
```

```
R2I = 0.2000
```

Lyapunov Iterations for the Coupled Nash Algebraic Riccati Equations

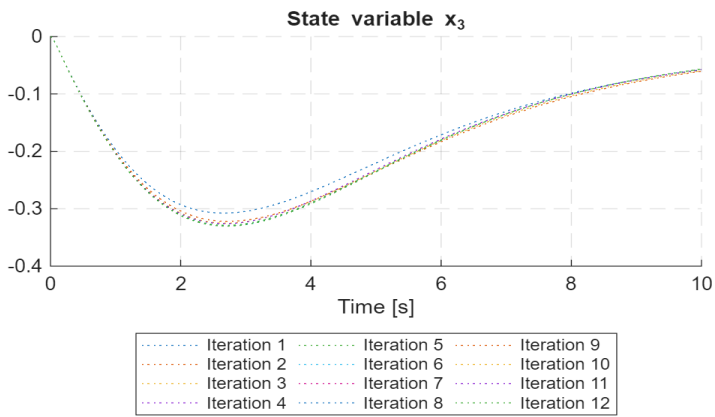
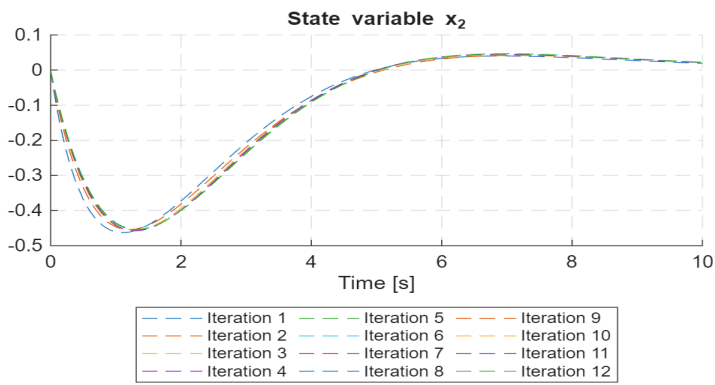
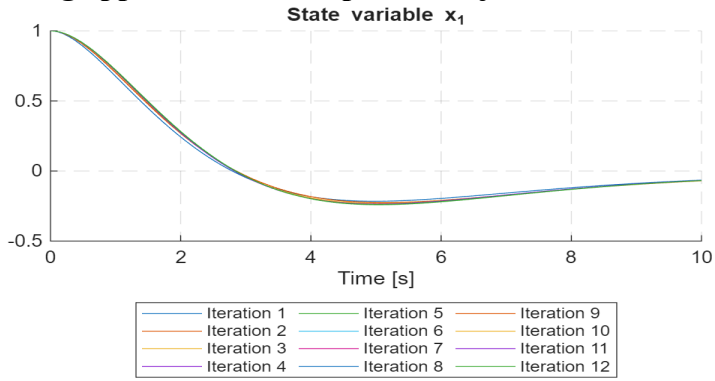
```
iter = 12
```

Calling Simulink

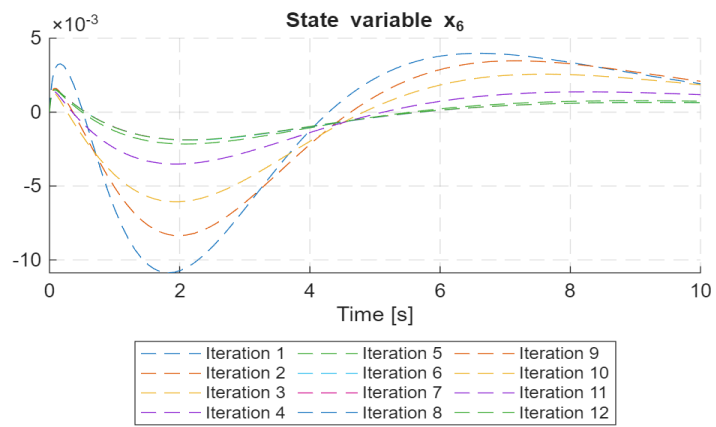
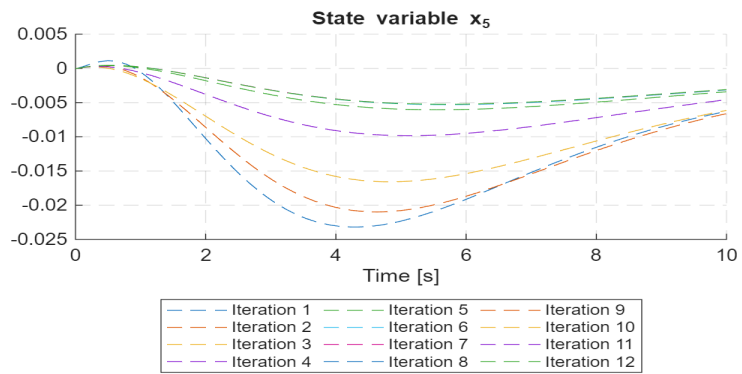
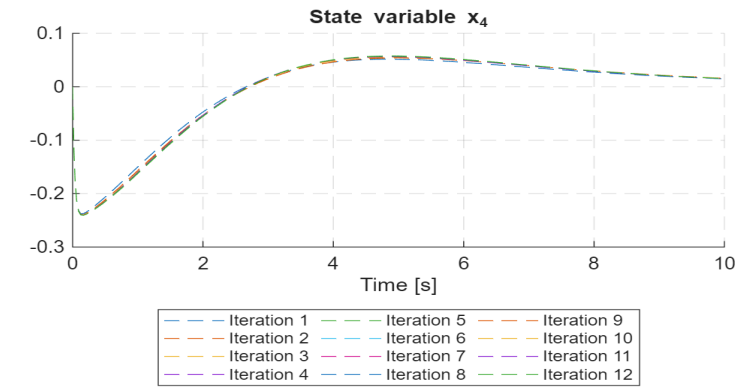
```
model =
```

```
Simulink.SimulationOutput:
```

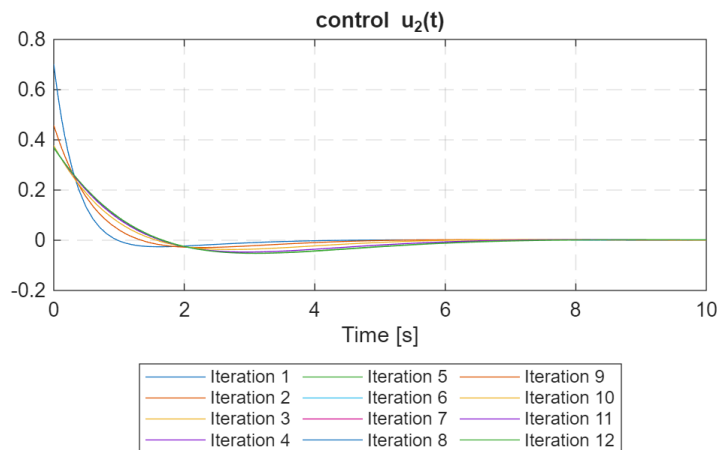
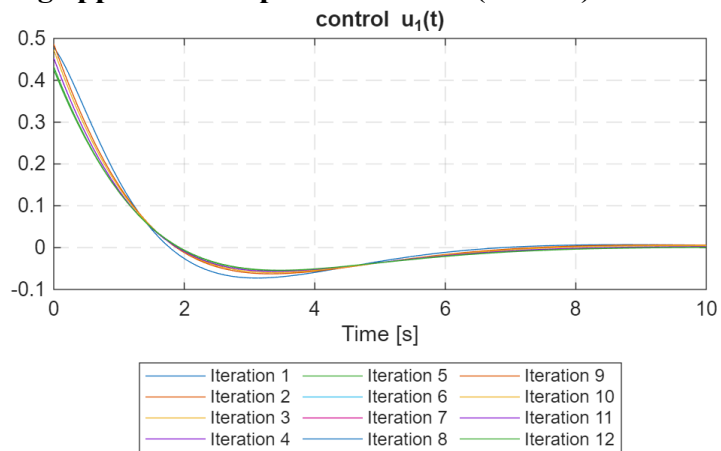
Plotting approximate and optimal trajectories



REINFORCEMENT LEARNING FOR ENGINEERING MATLAB DESIGN PROJECTS



Plotting approximate optimal controls (actions)



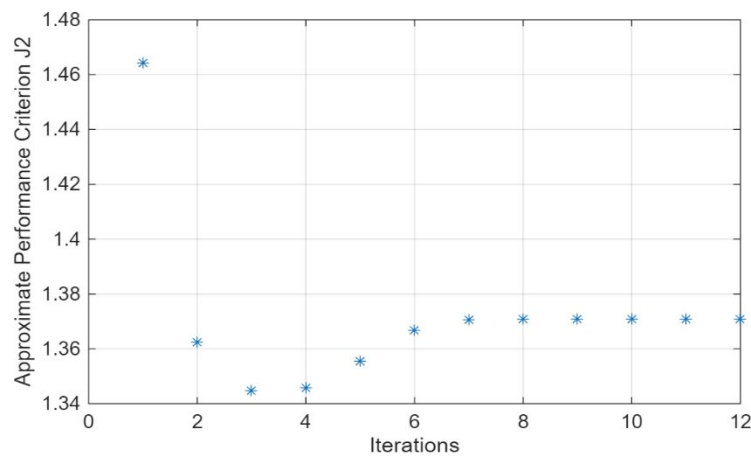
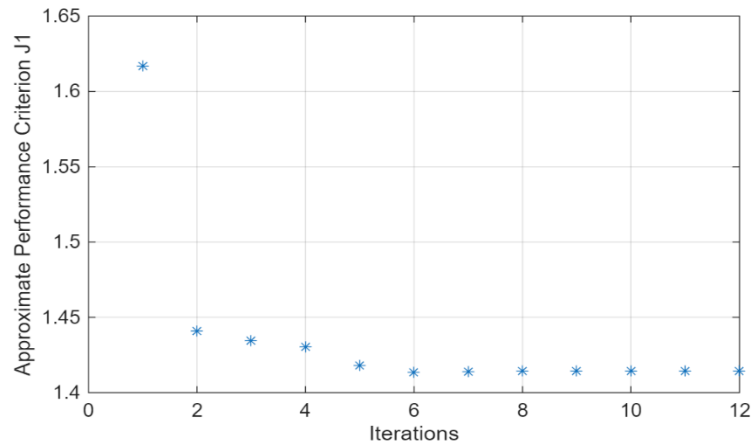
Solutions accuracy

$i = 12$

error1 = 1.2247×10^{-5}

error2 = 3.5641×10^{-5}

Plots of approximate and optimal performance criteria



error1 = 1.2247e-05

error2 = 3.5641e-05

7.5 References

- [1] A. Starr and Y. Ho, "Non-zero sum differential games," *Journal of Optimization Theory and Applications*, Vol. 3, 184-206, 1969a.
- [2] A. Starr and Y. Ho, "Further properties of non-zero sum differential games," *Journal of Optimization Theory and Applications*, Vol. 3, 207-219, 1969b.

- [3] R. Bellman, *Dynamic Programming*, Princeton University Press, Princeton NJ, 1957. Republished by Dover Publications Inc., Mineola NY, 2003.
- [4] T. Y. Li and Z. Gajic, "Lyapunov iterations for solving coupled algebraic Riccati equations of Nash differential games and the algebraic Riccati equation of zero-sum games," p. 333-351, in *New Trends in Dynamic Games and Applications*, G. J. Olsder (ed.), Birkhauser, 1995.
- [5] D. Vrabie and F. Lewis, "Integral Reinforcement Learning for Finding Online the Feedback Nash Equilibrium of Nonzero-Sum Differential Games," p. 313-330, Chapter 17 in *Advances in Reinforcement Learning*, A. Mellouk (ed.), IntechChina, 2012.
- [6] M. D. S. Aliyu, "An iterative computational scheme for solving the coupled Hamilton-Jacobi-Isaacs equations in non-zero differential games of affine nonlinear systems," *Decisions in Economics and Finance*, doi: 10.1007/s10203-017-0184-x, 2017.
- [7] Y. Chai, J. Luo, N.Han, and J. Xie, "Linear quadratic differential game approach for attitude takeover control of failed spacecraft," *Acta Astronautica*, Vol. 175, 142-154, 2020.
- [8] Y. Chai, J. Luo, N.Han, and J. Sun, "Robust event trigger game-based attitude control for an-orbit assembly," *Aerospace Science and Technology*, Vol. 103, 10584, Aug. 2020.
- [9] R. Kamalapurkar, J. Klotz, and W. Dixon, "Concurrent learning-based approximate feedback-Nash equilibrium solution of N -player nonzero-sum differential games," *IEEE/CAA Journal of Automatica Sinica*, Vol. 1, 239-247, 2014.
- [10] X. Li, Z. Peng, L. Liang, and W. Zha, "Policy iteration based Q-learning for linear nonzero-sum quadratic differential games," *Science China: Information Science*, Vol. 62, 052204:1-052204:19, May 2019.
- [11] E. Mazumdar, L. Ratliff, and S. Sastry, "On gradient-based learning in continuous games," *SIAM Journal on Mathematics of Data Science*, Vol. 2, 103-131, 2020.
- [12] K. Vamvoudakis, "Non-zero sum Nash Q-learning for unknown deterministic continuous-time linear systems," *Automatica*, Vol. 61, 274-281, 2015.
- [13] K. Vamvoudakis and F. Lewis, "Multi-player non-zero-sum games: Online adaptive learning solution of coupled Hamilton-Jacobi equations," *Automatica*, Vol. 47, 1556-1569, 2011.
- [14] D. Vrabie and F. L. Lewis, "Integral Reinforcement Learning for Online Computation of Nash Strategies on Nonzero sum Differential Games," p. 330-349, in *Reinforcement Learning and Approximate Dynamic Programming for Feedback Control*, F. Lewis and D. Liu (eds.), Wiley, 2013.
- [15] D. Vrabie, K. Vamvoudakis, and F. Lewis, *Optimal Adaptive Control and Differential Games by Reinforcement Learning Principles*, IET, London, United Kingdom, 2013.
- [16] M. Yang, Y-H. Luo, and Y-H. Wang, "Policy iteration algorithm for non-zero-sum games with unknown models," *Journal of Northeastern University*, Vol. 38, 318-321, 2015.
- [17] S. Yasini, M.B. Sitani Naghibi, and A. Kirampor, "Reinforcement learning and neural networks for multi-agent nonzero sum games of nonlinear constrained-input systems," *International Journal of Machine Learning and Cybernetics*, Vol. 7, 967-980, 2016.
- [18] Z. Zhang, J. Xu, and M. Fu, "Q-learning for feedback Nash strategy of finite-horizon nonzero-sum difference games," *IEEE Transactions on Cybernetics*, in press, 2021.
- [19] K. Vamvoudakis, H. Modares, B. Kiumarsi, and F. Lewis, "Game theory-based control system algorithms with real-time reinforcement learning: How to solve multiplayer games on line," *IEEE Control Systems Magazine*, 33-52, February 2017.

REINFORCEMENT LEARNING FOR ENGINEERING MATLAB DESIGN PROJECTS

- [20] D. Kleinman, “On an iterative technique for Riccati equations computations,” *IEEE Transactions on Automatic Control*,” Vol. 13, 114-115, 1968.1968.
- [21] Xu, L., *Learning and Control in Trustworthy and Responsible Artificial Intelligence for Cyber-Physical Systems*, Doctoral Dissertation, Rutgers University, 2024.
- [22] Xu, L., J. L. Muro, and Z. Gajic, “A new ADP reinforcement learning method for discrete-time LQ optimal control problems, *ASME Letters in Dynamic Systems and Control*, Vol. 148, 0110081 – 011081-8, Jan. 2026.